Homework 1

Total Points: 100 (Correctness and Style)

Due: Monday, April 7th, 11:59 pm

Starter Files

Download hw01.zip Inside the archive, you will find starter files for the questions of this homework. You cannot import anything to solve the problems.

Part 0: Integrity of Scholarship Agreement

Before starting the homework, please carefully read and fill out this integrity of scholarship agreement form. You **will NOT receive scores** in this class until you submit the form.

Click here to sign the form

Part 1. Coding and Docstring Style

Starting from HW1 we are going to check the style of your code. Most of the checks will be done automatically and you can check your code before submitting it.

Oftentimes, your code will be shared among other people, you wouldn't want them to find your code hard to use or understand. This becomes crucial especially when you get into the industry during your internship or full-time, and your fellow developers will demand good styling from your code. This is why we assign this homework at first to help you build the habit of enforcing styling.

In order to keep everything in order, the Python community has agreed on *recommended* programming styles to help everyone write code in a common style that makes the most sense for shared code. This style is captured in PEP-8. We are not going to enforce all requirements only the most common ones:

There are two ways you could check for the style:

1) Upload your code to Gradescope and our script will tell you what is missing/incorrect. **Note, there are still a few requirements that will be graded manually.**

2) Uploading code to Gradescope each time you need to test something is too annoying. Instead, you can follow the steps below and check the style of your code as you write it.

Link to style guide on the course website (the style guide is also located below)

Style Requirements

You will be graded for the style of programming on all homework assignments. A few key requirements for style are given below:

This is a complete Python style guide: Python Style Guide.

However, we will only test you on the smaller set of rules below:

- **Illegal Import Statements**: You should not import any package unless instructed to;
- **Module Docstring**: Every file that you submit should have a module docstring at the very top. In our assignments, it means to fill in name and PID;
- **Method Docstring**: Every method you create should have a docstring (i.e. method description)
    - Each docstring is surrounded by triple quotes (""") instead of triple single quotes (''')
    - This includes any inner function and helper function;
    - You don't need docstrings for lambda functions;
    - You may replace the entire # box with your method description;
    - The description should briefly describe what the method does, instead of what steps you take to achieve the result. Example:
        - Correct: Takes in a list of numbers, doubles each of them, and returns the doubled list;
        - Incorrect: Initializes an empty list and loops through the input list. For each number, I double it and append to the resulting list.
    - It's recommended to include input argument type and information like the examples given in lab01, but not required.
- **Line Limit**: All lines should be limited to a maximum of 79 characters.
    - Setup rulers in your editor
        - Sublime Text (reference): Go to *Preferences* > *Settings - User*, and add a new line in {}

          ```
          "rulers": [80]
          ```

        - You may also google "<editor name> set ruler" for your editor of choice
    - You may use backslash (\) to break up lines that might overflow:

      ```
      with open('/path/to/some/file/you/want/to/read') as file_1, \
           open('/path/to/some/file/being/written', 'w') as file_2:
          file_2.write(file_1.read())
      ```

    - You should follow the same rule in the docstring part. For expected doctest results, remove the leading whitespaces:

```python
def foo():
    """
    # Correct
    >>> foo()
    [[1, 2, 3], [1, 2, 3], \
[1, 2, 3]]

    # Incorrect
    >>> foo()
    [[1, 2, 3], [1, 2, 3], \
    [1, 2, 3]]
    """
    result = []
    for _ in range(3):
        result.append([1, 2, 3])
    return result
```

- **Magic Numbers**: Avoid using magic numbers (i.e. any number except 0, 1, -1) directly
  - If you need to use, say, 17 in your code, create a variable with a meaningful name and use the variable instead
  - The reason behind is that numbers have meanings. 17 might mean distance in some context and age in another context. You should define meaningful variable names to differentiate between the meanings.
  - Do not use a variable name like "SEVENTEEN" because it does not define the meaning clearly, so it will be considered as meaningless variable name (see the next rule)
  - <u>Exceptions</u>: We will not deduct points if magic numbers appear in your code in the following scenarios

ALLOWED MAGIC NUMBER EXCEPTIONS
1. Checking for mod n, where n can be any number. You don't have to make a variable for the number n.
`number_to_check % 2 == 0 # Example for mod 2` `number_to_check % 3 == 0 # Example for mod 3`
2. The distance formula (2D & 3D examples are given, but the distance formula is ok for any dimension N).
`a ** 2 = b ** 2 + c ** 2 # 2D distance formula` `d ** 2 = x1 ** 2 + x2 ** 2 + x3 ** 2 # 3D distance formula`
3. Root formulas such as square root, cube root etc:
`square_root = a_number ** (1/2)` `cube_root  = a_number ** (1/3)`

- **Meaningless Variable Names**: All variable and function names should be descriptive

    - Function names typically evoke operations applied to arguments by the interpreter (e.g., print, add, square) or the name of the quantity that results (e.g., max, abs, sum);
    - Parameter names should evoke the role of the parameter in the function, not just the kind of argument that is allowed. For example, if the variable stores a list of student names, then it could be called student_names instead of lst;
    - Single letter parameter names are acceptable when their role is obvious, but avoid "l" (lowercase ell), "O" (capital oh), or "I" (capital i) to avoid confusion with numerals;
    - Try to only use i, j, k as index names and avoid them for other uses;
    - Avoid using built-in function names as variable names anywhere, as they break the built-in functions. For example, you should not use sum, dict, map, etc. as variable names.
- **Bad Variable Style**: You should always use snake_case when coding in Python
    - Variable and function names are lowercase, with words separated by underscores, e.g. unusual_sum()
    - Single-word names are preferred.
- **Indentation**: All indentations MUST be 4 spaces instead of TABs.
    - You can automatically convert tabs into spaces in the settings of your editor. Search for the editor setting “soft tabs” and set the soft tab length to 4;
    - Another indication is that when you upload to gradescope, tabs will have length 8 while 4 spaces will only have length 4.
- **Doctests**: For EACH function created, you should add **at least 3 doctest of your own** unless explicitly instructed otherwise.
    - You should think comprehensively about cases that would possibly break your code rather than testing it on easy-to-pass scenarios;
    - If you create any helper functions, you still need to add docstrings/doctests for them;
    - If you create inner functions, you only need to add docstrings (no doctests required);
    - It’s recommended to add as many doctests as you’re comfortable with to test the code thoroughly because your code correctness will be graded by hidden tests.

To further clarify the rules, we have compiled a set of example problems and solutions with an emphasis on style requirements. Feel free to ask any questions via Edstem.

Style Guide Examples

Submission

By the end of this homework, you should have submitted the homework via Gradescope. You may submit more than once before the deadline; only the final submission will be graded. Refer to Lab00 directions for submission.

Testing

At any point of the homework, use the following command to test your work:

```
>>> python3 -m doctest hw01.py
```

Part 2. Important Requirements and Questions

1. **DO NOT IMPORT ANY PACKAGES.**
2. **You should only use BASIC LOOPS (for/while) and/or CONDITIONAL STATEMENTS (if/elif/else) in your method structures. Specifically, you should NOT use map(), filter(), zip(), reduce(), or lambda functions to solve the questions for this homework. Other built-in methods are allowed. You can ask on Ed if you have any doubts.**
3. Add your own doctests (at least three per function) as the given doctests are not sufficient. **You will be graded on the doctests**. The doctests encompass **5 points** of your submission, so make sure you add them to each function!
4. In this assignment you can assume that **the input given to you is valid and does not have any errors**.
5. Make sure you abide by the style guide outlined above. Style encompasses **10 points** of your submission, so make sure you follow it.
    a. Remember to add the docstring descriptions. They should be a brief description of the problem in your own words. Don't just copy and paste from the writeup.
    b. Remember to define magic numbers (anything other than -1,0,1) as variables.
    c. Remember to keep each line of code under 80 characters. This applies to the docstrings and doctests as well. You can define a ruler at 79 characters in your editor to make it easier to see if the length goes over.
6. **For this homework**: You can assume that we will test on valid inputs. For example, if a function takes a positive integer we will only test your code on positive integers.

Question 0:

Quiz 0 (1 point)

Log in to Canvas and take Quiz 0 to refresh your understanding of the syllabus (you can find it on our website: dsc20.org). Make sure you understand how to find and take it. All reading quizzes will be there. The deadline for this quiz is the same as Homework 1's deadline.

Question 1:

Make sure that you completed part 0. Your assignment will NOT be graded without it.

Your preparation to open up your own business continues. All members of the team need to create their own logins.

Write a function that takes two parameters:

- `fname`, as a string
- `lname`, as a string

It then returns a login as *string* constructed in the following way:

- `fname` is reversed and then every other character is taken (starting from index 0)
- Every third character is taken from the `lname` (starting from index 0)

The resulting characters are concatenated to create a login as a string, then return it.

Requirement: only basic loops (for/while) and/or conditional statements (if/elif/else)

```
def login(fname, lname):
    """
    >>> login("Marina", "Langlois")
    'aiaLgi'
    >>> login("", "")
    ''
    >>> login("San", "Diego")
    'nSDg'
    """
```

Question 2:

As we already know from lab01, you need to have someone 23 or older for renting an office. You'd like to gather more information about people's ages.

Write a function that takes two positive integers and returns whichever age is closest to but still strictly less than 23. Return *"You both can rent!"* if both ages are at least 23.

Example:

Input	Output	Reason
`ages(19, 21)`	21	21 is closer to 23 than 19.
`ages(26, 21)`	21	21 is closer to 23, since 26 is already over.

```
def ages(age1, age2):
    """
    >>> ages(19, 21)
    21
    >>> ages(26, 21)
    21
    >>> ages(26, 27)
    'You both can rent!'
    >>> ages(19, 23)
    19
    """
```

Question 3:

On your team there are multiple people aged 23 or older, who are eligible to rent the space. To decide who will handle the contract, your team agreed to assign it based on name length.

Write a function that takes in three names (as strings) and returns the **longest** one. If there is a tie, you should return the name that appears **last** in the parameter list.

```
def renter(name1, name2, name3):
    """
    >>> renter("K", "BB", "Joy")
    'Joy'
    >>> renter("Joy", "K", "BB")
    'Joy'
    >>> renter("BB", "Joy", "K")
    'Joy'
    >>> renter("BB", "K", "Jo")
    'Jo'
```

```
>>> renter("BB", "Jo", "Su")
'Su'
"""
```

Question 4:

Turns out, there are many buildings you can rent from, so other factors need to be considered. One of them is the proximity to good lunch spots, which shouldn't be too far away.

Write a function that takes 4 parameters:

- `lunch_places`: a list of lists, where each list contains 2 floating point numbers (representing coordinates of a place of a lunch place).
- `office_x, office_y`: two floating point numbers representing the location of the starting point.
- `threshold`: positive number, maximum distance you are willing to walk.

Your function should return a list of coordinates of lunch places, where the distance between each lunch place and the office is within the threshold.

Input	Output	Reason
[[0, 0], [30.5, 20.7]], 3.2, 4, 6	[[0, 0]]	* The distance from [0, 0] to 3.2, 4 is less than or equal to 6, so we keep [0,0] * The distance from [30.5, 20.7] to 3.2, 4 is greater than 6, so we discard it.
[[100, 100]], 100.5, 100, 0.2	[]	* The distance between [100, 100] and 100.5, 100 is greater than the given threshold (0.2), so we discard it.

It is often useful to create *helper* methods for your primary function. This way you use one function to do one job and it can be reused if needed. Let's see how we can do it.

Question 4.1:

Write a (helper) function that calculates (and returns) the Euclidean distance between two given points. Notice that the parameter list is different from the original question.

- `lst`: a list containing 2 floating point numbers (representing coordinates)
- `x2, y2`: two floating point numbers representing the location of the starting point

```
def helper_distance(lst, x2, y2):
    """
    >>> helper_distance([0, 0], 3, 4)
    5.0
    >>> helper_distance([-3, -4], 3, 4)
    10.0
    >>> helper_distance ([100, 100], 100.5, 100)
    0.5
    """
```

Question 4.2:

Now using your function from 4.1, solve the original question. In general, try to break a problem into smaller steps, write shorter helper functions and use them to solve a more complex problem.

```
def lunch(lunch_places, office_x, office_y, threshold):
    """
    >>> lunch([[0, 0], [30.5, 20.7]], 3.2, 4, 6)
    [[0, 0]]
    >>> lunch([[-3, -4], [6, 7]], 3, 4, 10)
    [[-3, -4], [6, 7]]
    >>> lunch ([[100, 100]], 100.5, 100, 0.2)
    []
    """
```

Question 5:

Let's expand on Question 4 by assigning each coordinate to a restaurant. Instead of showing coordinates within a threshold, we want to see the names of places we can walk to.

Your new function takes 5 parameters this time:

- `lunch_places`: a list of lists, where each list contains 2 floating point numbers (representing coordinates of a place of a lunch place),

- `office_x, office_y`: two floating point numbers representing the location of the starting point.
- `threshold`: positive number, maximum distance you are willing to walk.
- `names`: a list of strings, where each string represents the name of a place for the corresponding coordinate in a `lunch_places`.

The lengths of two lists are the same.

Input	Output	Reason
[[0, 0], [30, 20], [5, 9]], 3.2, 4, 6, `['place1', 'place2', 'place3']`	`['place1', 'place3']`	* The distance from [0, 0] to 3.2, 4 is less than 6, so we keep the name of this place, `place1`. * The distance from [30, 20] to 3.2, 4 is greater than 6, so we discard it. * The distance from [5, 9] to 3.2, 4 is less than 6, so we keep the name of this place, `place3`.

```
def lunch_names(lunch_places, office_x, office_y, threshold, names):
    """
    >>> lunch_names([[0, 0], [30, 20], [5, 9]], 3.2, 4, 6, \
    ['place1', 'place2', 'place3'])
    ['place1', 'place3']
    >>> lunch_names([[-3, -4], [6, 7]], 3, 4, 10, \
    ['place1', 'place2'])
    ['place1', 'place2']
    >>> lunch_names ([[100, 100]], 100.5, 100, 0.2, ['place1'])
    []
    """
```

Question 6:

After the weekend, the team decided to meet in person, and the invitation needs to be sent out.

Write a function that takes in the name of the invitee, time of day, place and the name of the message creator, as strings. It then returns an invitation, as a string. Assume all inputs are valid, but they may include an empty string.

When you print the returned string to the console, it should look like this:

Dear <*name*>, Please join our meeting at <*time*>, at the <*place*>. See you soon: <*sender's name*>

*(**Note:** The token <BLANKLINE> in the doctest denotes a blank line in the output. You should not append this token in the returned string. You should always add this token in your doctest if you expect the output to have a blank line, instead of just hitting the Enter key.)*

```
def meeting_message(i_name, time, place, s_name):
    """
     >>> print(meeting_message("Penny", "3:15pm", "Cheesecake Factory",
\
    "Sheldon"))
    Dear Penny,
    Please join our meeting at 3:15pm, at the Cheesecake Factory.
    <BLANKLINE>
    See you soon: Sheldon

    >>> print(meeting_message("Freya", "", "Dog Park", "Marina"))
    Dear Freya,
    Please join our meeting at , at the Dog Park.
    <BLANKLINE>
    See you soon: Marina
    """
```

Question 7:

Since the meeting is in-person, every participant should get an assigned seat number. Write a function that takes a list of team member names (as strings) and returns another list, where each element is the length of the name from the given list. Unfortunately, we can not assign the same number more than once, so if there are names of equal length then assign "taken" instead.

Hint: built-in operator `in` might be helpful.

```
def seat_number(lst):
    """
    >>> seat_number(["Marina", "Tom", "B"])
    [6, 3, 1]
```

```
    >>> seat_number(["Marina", "Sue", "Ben", "Freya"])
    [6, 3, 'taken', 5]
    >>> seat_number(["Marina", "Sue", "Ben", ""])
    [6, 3, 'taken', 0]
    """
```

Question 8:

One important decision to be made is the type of the computers for the offices. Everyone took a vote and it was recorded in a list. People had to choose between desktops and laptops.

Write a function that takes a list of choices (as strings) and calculates whether the desktop computer `DESKtop` occurs more often than the type `LAPtop` (i.e. the count of DESKtop is larger).
Return True if `DESKtop` occurs more often and False otherwise.

Notes:

- It's not guaranteed that `DESKtop` and/or `LAPtop` will appear in the list, and there might be other types as well.
- The strings are case-sensitive, meaning that we **only** consider `DESKtop` instead of `desktop`, `DESKTOP`, etc.

Hints:

1. You may want initialize variables that track the numbers for 'DESKtop' and 'LAPtop';
2. Another option is to use a built-in function `count()`

```
def computers(choices):
    """
    >>> computers(["DESKtop", "LAPtop", "DESKtop"])
    True
    >>> computers(["LAPtop", "LAPtop"])
    False
    >>> computers(["DESKtop", "Pager", "Tablet", "LAPtop"])
    False
    """
```

Question 9:

During the meeting your friends became interested in the average age of the team. Everyone recorded their age (as a string) but some did not want to share their age, so they used a negative number (also as a string).

Write a function that takes a list of strings (representing ages), and returns the age average (as a string) of only **positive** numbers, rounded to one decimal place. Return "0.0" (as a string) if the input list is empty or has no positive numbers.

Assumption:

- The strings inside the list input can't be empty.

```
def age_average(lst):
    """
    >>> age_average(["20", "21", "22"])
    '21.0'
    >>> age_average(["50", "25", "30"])
    '35.0'
    >>> age_average(["40", "-999", "45"])
    '42.5'
    >>> age_average([])
    '0.0'
    """
```

Question 10:

Painters are about to start their job but they need constant supervision.

Write a function that takes a list of participants (as strings) and a name of the painting company. It then creates two teams:

- The first team is made up of participants from every even index of the given list.
- The second team is made up of participants from every odd index of the given list (opposite to the first team)

You need to save these names into two different lists, also each list should contain the company name: the first team has it as the first element, the second team has it as the last element. See doctests for examples.

```
def supervision_teams(team, company_name):
    """
    >>> supervision_teams(["p1", "p2", "p3"], "Marina")
    (['Marina', 'p1', 'p3'], ['p2', 'Marina'])
    >>> supervision_teams(["p1"], "Marina")
```

```
    (['Marina', 'p1'], ['Marina'])
    >>> supervision_teams(["p1", "p2", "p3", "p4", "p5", "p6"],
"Marina")
    (['Marina', 'p1', 'p3', 'p5'], ['p2', 'p4', 'p6', 'Marina'])
    """
```

Submission

By the end of this homework, you should have submitted `hw01.py` via Gradescope. You may submit more than once before the deadline; only the final submission will be graded.

Important Notes:

1. You may submit more than once before the deadline; only the final submission will be graded;
2. We will be grading this lab solely based on hidden tests, so you should test your code thoroughly by writing more test cases (encouraged to go over the required 3, add as many as you like);
3. Unlike lab00, you should expect to see "-/100.0". You will see your score when the grades are released;
4. You should **wait for the Autograder to finish** running before leaving the site to ensure that the tests are properly run;
5. If your autograder fails to run, try to consult the Gradescope Common Errors section on the course website before making an ED post;
6. If you have any other questions, please post to ED.

Warning: If you used Visual Studio Code as your editor, please double check if there are weird import statements that you don't recognize before submitting. Make sure to remove these as they will fail Gradescope runs!